Bài 1: Chat Server

Mã nguồn:

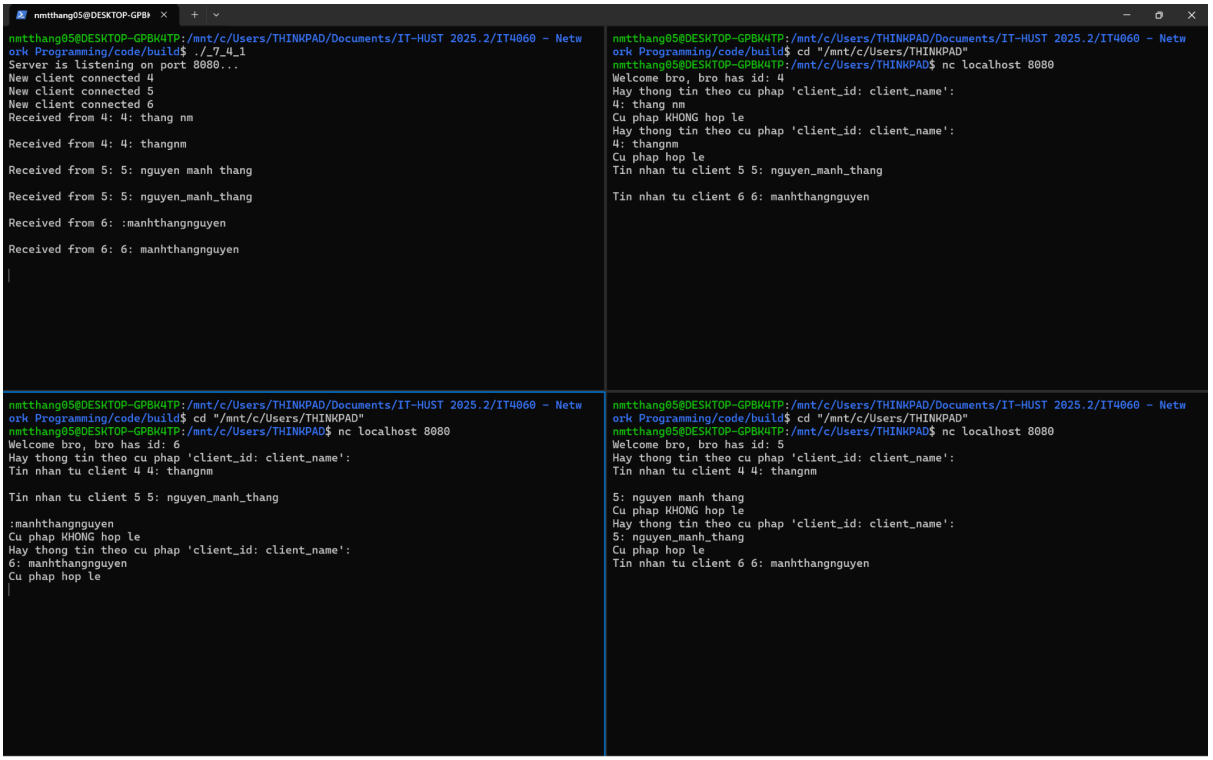
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <netdb.h>
#include <sys/select.h>

typedef struct {
    char id[256];
    char content[256];
    char timestamp[256];
}msg;

int check_format(char* buf, msg* client_msg){
    char* s = strtok(buf, " ");

    if(s == NULL){
        return 0;
    }

    if(s[strlen(s)-1] != ':'){
        return 0;
    }

    s[strlen(s)-1] = 0;
    if(strlen(s) <= 0){
        return 0;
    }

    memcpy(client_msg->id, s, strlen(s)+1);

    s = strtok(NULL, " ");
    char* s1 = strtok(NULL, " ");
    if(s1 != NULL){
        return 0;
    }

    memcpy(client_msg->content, s, strlen(s)+1);
```

```
    return 1;
}

int main(){
    int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &(int){1},
sizeof(int))) {
        perror("setsockopt() failed");
        close(listener);
        return 1;
    }

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(8080);

    if(bind(listener, (struct sockaddr*)&addr, sizeof(addr))){
        perror("bind() failed");
        close(listener);
        return 1;
    }

    if(listen(listener, 5)){
        perror("listen() failed");
        close(listener);
        return 1;
    }

    printf("Server is listening on port 8080...\n");

    char* hello_msg = "Welcome bro, bro has id:";
    char* request_msg = "Hay thong tin theo cu phap 'client_id:
client_name':\n";
    char* warning = "Cu phap KHONG hop le\n";
    char* valid_syntax = "Cu phap hop le\n";

    fd_set fdread, fdtest;
    FD_ZERO(&fdread);
    FD_ZERO(&fdtest);
    FD_SET(listener, &fdread);
```

```
    char buf[256];
    while(1){
        fdtest = fdread;

        int ret = select(FD_SETSIZE, &fdtest, NULL, NULL, NULL);
        if(ret < 0){
            perror("select() failed");
            break;
        }

        if(ret == 0){
            printf("Timed out\n");
            continue;
        }

        for(int i = 0; i < FD_SETSIZE; ++i){
            if(FD_ISSET(i, &fdtest)){
                if(i == listener){
                    int client = accept(listener, NULL, NULL);
                    if(client < FD_SETSIZE){
                        printf("New client connected %d\n", client);

                        char id_norti[256];
                        snprintf(id_norti, sizeof(id_norti), "%s %d\n",
hello_msg, client);
                        send(client, id_norti, strlen(id_norti), 0);

                        send(client, request_msg, strlen(request_msg),
0);
                        FD_SET(client, &fdread);
                    }else{
                        printf("Oh shit, out of slots\n");
                        close(client); // luu y, neu khong close client
van se ton tai;
                    }
                }else{
                    ret = recv(i, buf, sizeof(buf), 0);
                    if(ret <= 0){
                        printf("Client %d disconnected\n", i);
                        FD_CLR(i, &fdread);
                    }else{
                        buf[ret] = 0;
```

```
                        printf("Received from %d: %s\n", i, buf);
                        msg client_msg;

                        if(check_format(buf, &client_msg)){
                            char i_msg[538];
                            snprintf(i_msg, sizeof(i_msg), "Tin nhan tu
client %d %s: %s\n", i, client_msg.id, client_msg.content);

                            for(int j = 4; j < FD_SETSIZE; ++j){
                                if(j != i && FD_ISSET(j, &fdread)){
                                    send(j, i_msg, strlen(i_msg), 0);
                                }
                            }
                            // printf("%s: %s\n", client_msg.id,
client_msg.content);
                            send(i,valid_syntax, strlen(valid_syntax),
0);
                        }else{
                            send(i, warning, strlen(warning), 0);
                            send(i, request_msg, strlen(request_msg),
0);
                        }
                    }
                }
            }
        }
    }
}
```

Hình ảnh minh họa:



Hình ảnh trên là minh họa 1 server và có 3 client 4, 5, 6 kết nối tới. Client 4 - góc trên bên phải, client 5 - góc dưới bên phải, client 6 góc dưới bên trái.

Use case thực hiện test:

- Client 4 gửi một message với cú pháp “4: thang nm” vi phạm cú pháp đặt ra do tên không được phép có dấu cách, server gửi về thông báo “Cu phap KHONG hop le” và không chuyển tiếp bất cứ message này cho client nào khác. Sau đó, client 4 gửi message với cú pháp “4: thangnm” hợp lệ với cú pháp, nên server gửi về thông báo “Cu phap hop le” và chuyển tiếp message này cho các client khác.
- Client 5 có tình huống test tương tự cho như Client 4.
- Client 6 gửi một message với cú pháp “: manhthangnguyen” vi phạm cú pháp đặt ra do không có id. Sau khi gửi message “6: manhthangnguyen” hợp lệ nên chuyển tiếp cho các client khác.

Bài 2: Telnet server

Mã nguồn:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
#include <unistd.h>
```

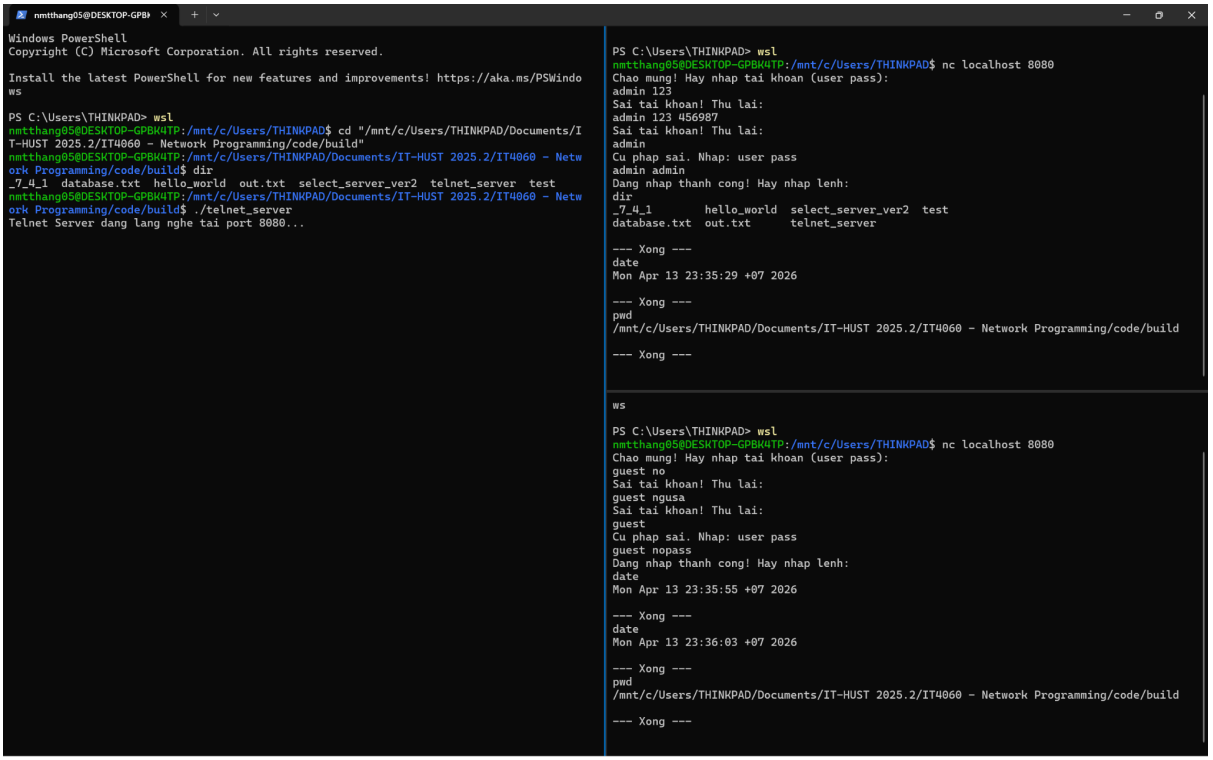
```c
#include <sys/select.h>

#define PORT 8080
#define BACKLOG 5
#define BUF_SIZE 256

int client_auth[FD_SETSIZE];

void remove_newline(char *s) {
    while (*s) {
        if (*s == '\r' || *s == '\n'){
            *s = '\0';
            break;
        }
        s++;
    }
}

int main() {
    for (int i = 0; i < FD_SETSIZE; i++) client_auth[i] = 0;

    int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (listener == -1) {
        perror("socket() failed");
        return 1;
    }

    int opt = 1;
    setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);

    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr))) {
        perror("bind() failed");
        return 1;
    }

    listen(listener, BACKLOG);

    printf("Telnet Server dang lang nghe tai port %d...\n", PORT);
```

```
    fd_set master_fds, read_fds;
    FD_ZERO(&master_fds);
    FD_SET(listener, &master_fds);
    int max_fd = listener;

    char buf[BUF_SIZE];

    while (1) {
        read_fds = master_fds;
        if (select(max_fd + 1, &read_fds, NULL, NULL, NULL) < 0) break;

        for (int i = 0; i <= max_fd; i++) {
            if (FD_ISSET(i, &read_fds)) {
                if (i == listener) {
                    int client = accept(listener, NULL, NULL);
                    FD_SET(client, &master_fds);
                    if (client > max_fd) max_fd = client;

                    char *welcome = "Chao mung! Hay nhap tai khoan
(user pass):\n";
                    send(client, welcome, strlen(welcome), 0);
                } else {
                    int ret = recv(i, buf, sizeof(buf) - 1, 0);
                    if (ret <= 0) {
                        close(i);
                        FD_CLR(i, &master_fds);
                        client_auth[i] = 0;
                    } else {
                        buf[ret] = '\0';
                        remove_newline(buf);

                        if (client_auth[i] == 0) {
                            char user[64], pass[64];
                            if (sscanf(buf, "%s %s", user, pass) == 2)
{
                                FILE *f = fopen("database.txt", "r");
                                if (f == NULL) {
                                    char *msg = "Loi server: khong thay
DB.\n";
                                    send(i, msg, strlen(msg), 0);
                                    continue;
                                }
```

```
                                char f_user[64], f_pass[64];
                                int found = 0;
                                while (fscanf(f, "%s %s", f_user,
f_pass) == 2) {
                                    if (strcmp(user, f_user) == 0 &&
strcmp(pass, f_pass) == 0) {
                                        found = 1; break;
                                    }
                                }
                                fclose(f);

                                if (found) {
                                    client_auth[i] = 1;
                                    char *msg = "Dang nhap thanh cong!
Hay nhap lenh:\n";
                                    send(i, msg, strlen(msg), 0);
                                } else {
                                    char *msg = "Sai tai khoan! Thu
lai:\n";
                                    send(i, msg, strlen(msg), 0);
                                }
                            } else {
                                char *msg = "Cu phap sai. Nhap: user
pass\n";
                                send(i, msg, strlen(msg), 0);
                            }
                        } else {
                            char cmd[BUF_SIZE + 20];
                            snprintf(cmd, sizeof(cmd), "%s > out.txt
2>&1", buf);
                            system(cmd);

                            FILE *f = fopen("out.txt", "r");
                            if (f) {
                                char line[BUF_SIZE];
                                while (fgets(line, sizeof(line), f)) {
                                    send(i, line, strlen(line), 0);
                                }
                                fclose(f);
                            }
                            char *msg = "\n--- Xong ---\n";
                            send(i, msg, strlen(msg), 0);
                        }
```

```
                        }
                    }
                }
            }
        }
    return 0;
}
```

Hình ảnh minh họa:



Use case thực hiện test:

- Với client ở góc trên bên phải, khi kết nối vào server sẽ được yêu cầu đăng nhập, nếu tài khoản password sai hoặc sai cú pháp sẽ bị yêu cầu nhập lại đến khi chính xác. Khi xác thực hoàn thành có thể thực hiện các lệnh bất kỳ như: dir - liệt các file/folder trong thư mục chứa file executable, date - đưa ra ngày giờ, pwd - đưa ra thư mục cha …
- Với client ở góc dưới bên phải, các trường hợp test được thực hiện tương tự như trên.

Link git: [Link](#).